

Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «Уральский федеральный университет имени первого
Президента России Б.Н. Ельцина»
Институт радиоэлектроники и информационных технологий-РТФ

ОТЧЕТ

По лабораторной работе №3

«Внедрение Dependency Injection и SQLAlchemy в Litestar»

По дисциплине «Разработка приложений»

Выполнил: Лапушкин И. А.

Группа: РИМ-150950

Проверил преподаватель:
Кузьмин Д. И.

Екатеринбург

2025

Цель работы: Освоить принципы Dependency Injection и интеграцию SQLAlchemy ORM в веб-приложении на базе фреймворка Litestar написав CRUD.

Задачи:

1. Настроить рабочее окружение.
2. Реализовать слой Repository для работы с базой данных
3. Реализовать сервисный слой с бизнес-логикой.
4. Создать контроллер для обработки HTTP-запросов.
5. Настроить Dependency Injection в главном приложении
6. Реализовать пагинацию с подсчетом общего количества

Ход работы:

1. Подготовка среды разработки

В pycharm создаем папку проекта, создаем и активируем виртуальное окружение, загружаем требуемые библиотеки, создаем контейнер с PostgreSQL, создаем необходимые файлы

Создали контейнер с БД:

```
docker run --name lr3-db -e POSTGRES_PASSWORD=1 -e  
POSTGRES_DB=lr3-db -p 5433:5432 -d postgres:15
```

2. Развертывание приложения

После запуска приложения наблюдаем, что всё работает (Рисунок 1).

```
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Рисунок 1 – Запуск команды «python main.py»

Далее переходим на <http://localhost:8000/schema>, (Рисунок 2)

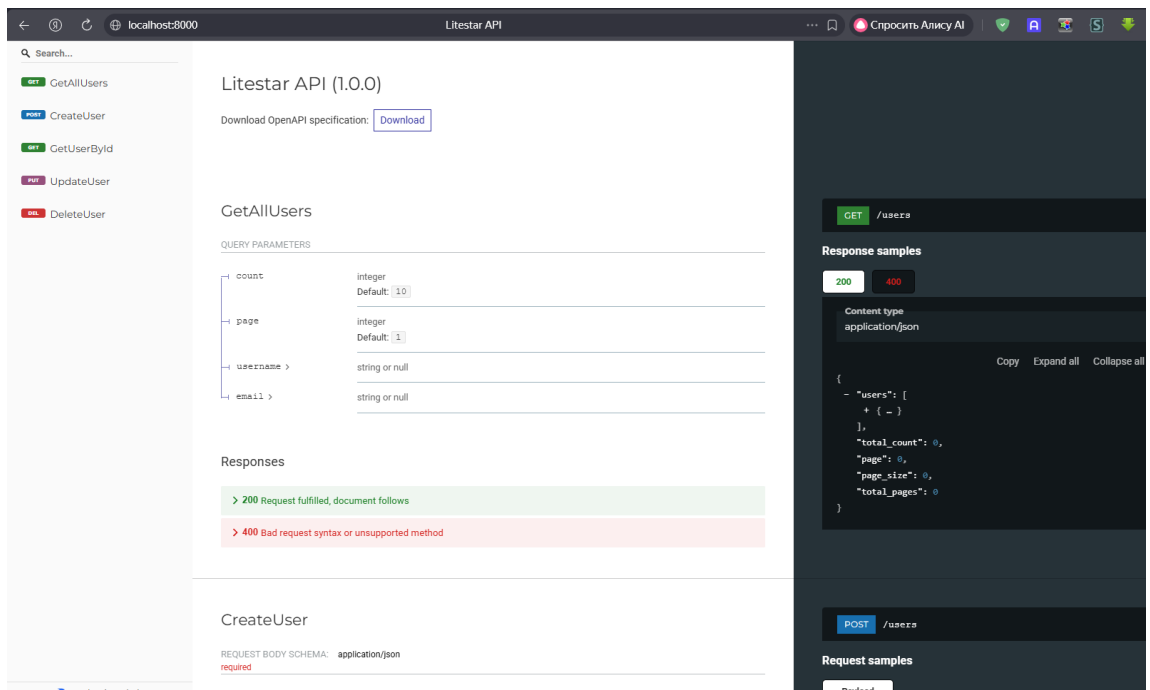


Рисунок 2 – Результат открытия «<http://localhost:8000/schema>»

Далее переходим на <http://localhost:8000/users>, видим наших пользователей из лабораторной работы 2

3. Тестирование CRUD-операций

Протестируем различные запросы. Как видим из рисунков 3-7, всё работает

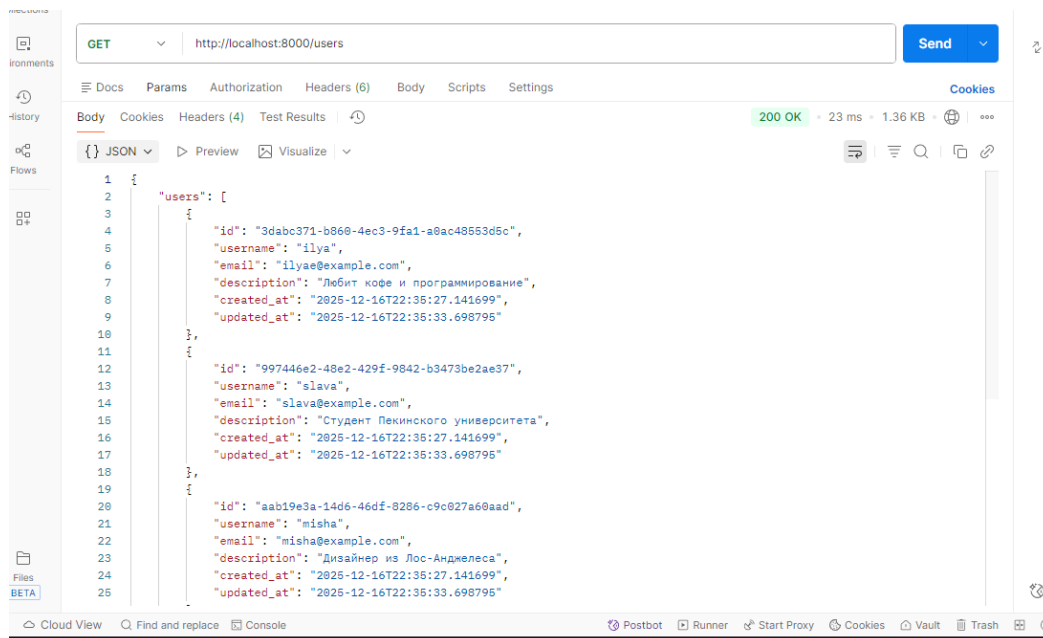


Рисунок 3 – Получение всех пользователей

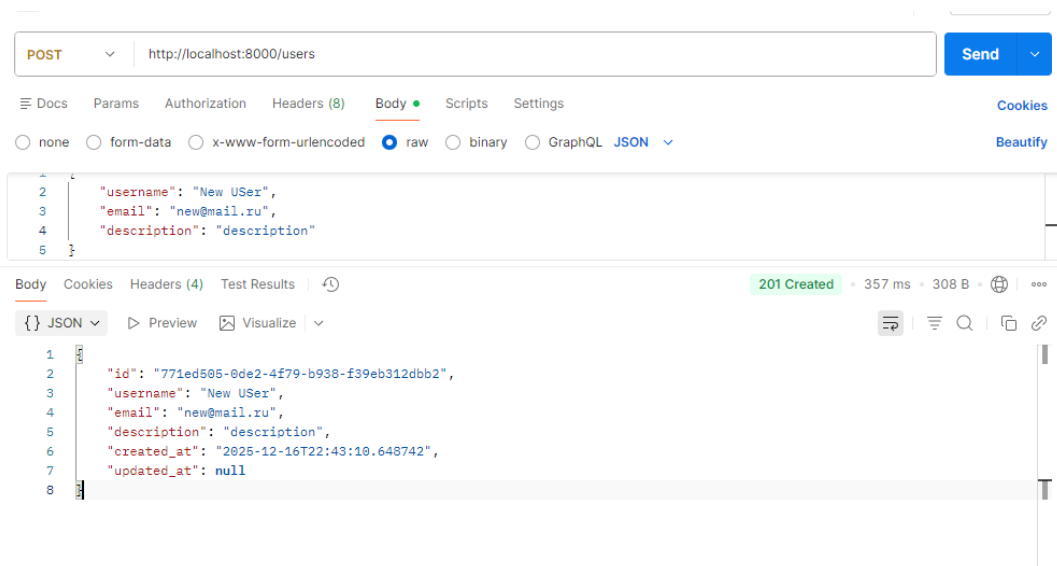


Рисунок 4 – Создание нового пользователя и проверка

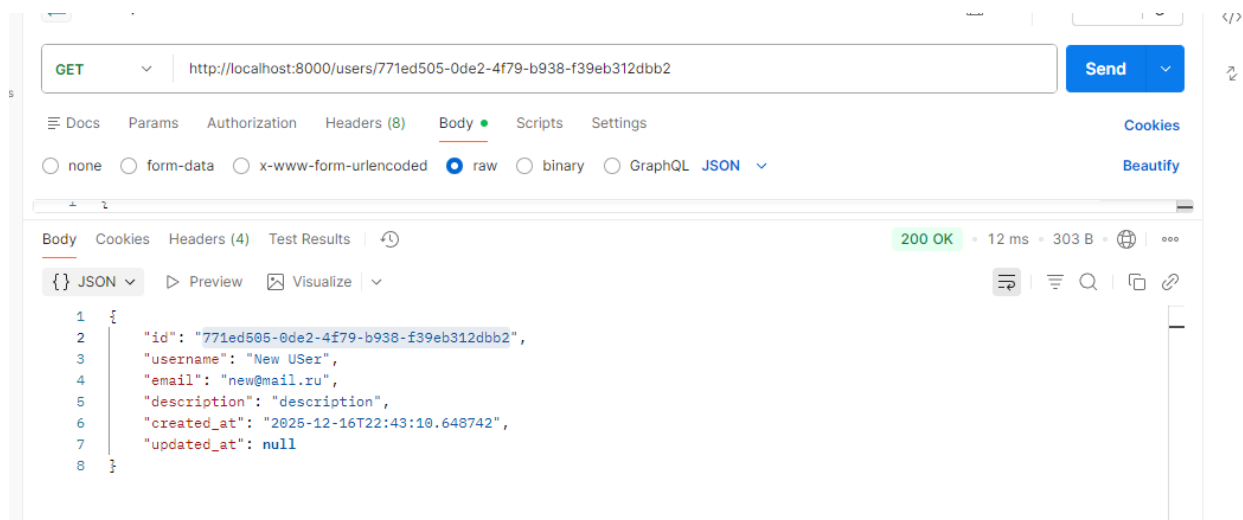


Рисунок 5 – Поиск пользователя по ID

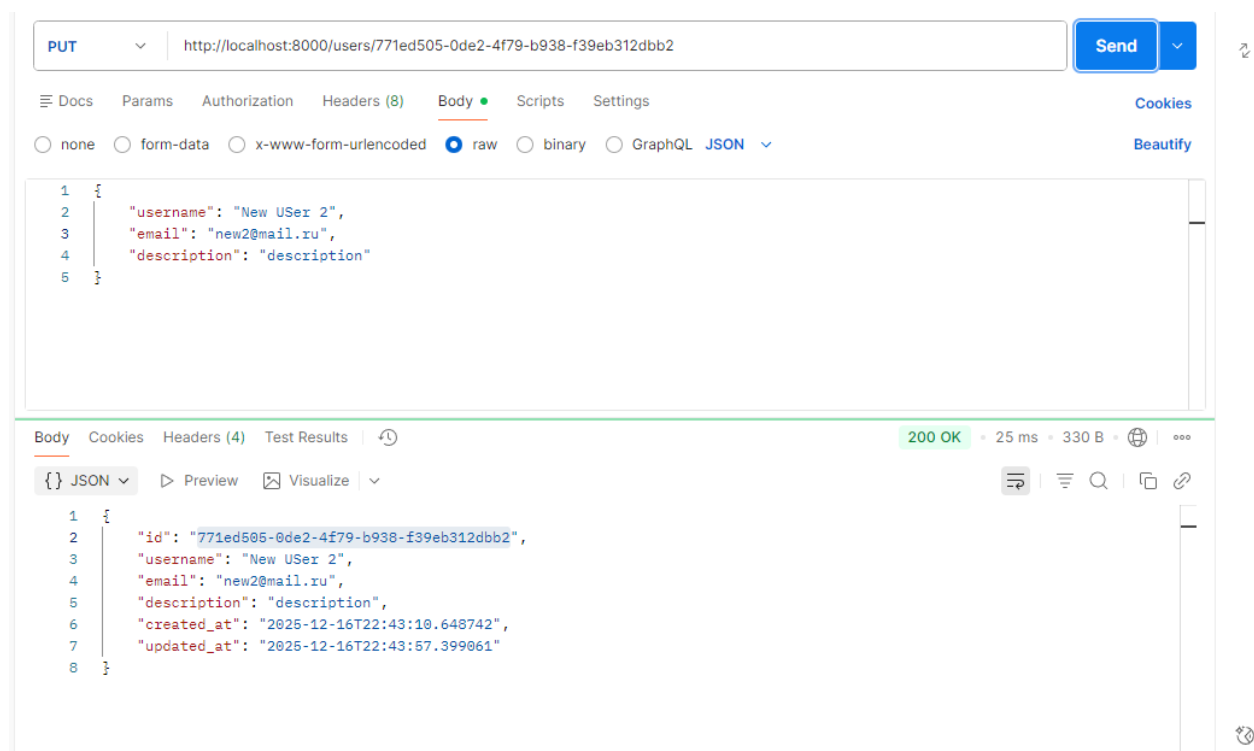


Рисунок 6 – Обновление пользователя

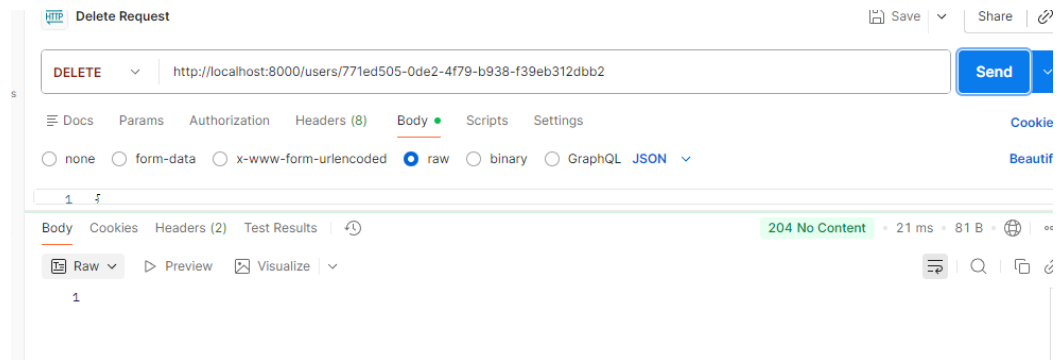


Рисунок 7 – Удаление пользователя

Вопросы:

1. Объясните принцип Dependency Injection (DI) своими словами. Какую проблему он решает в контексте разработки приложений и какие преимущества дает?

Dependency Injection – это подход, при зависимости (например, сервисы или репозитории) не создаются внутри классов, а передаются извне, обычно через конструктор. Это решает проблему тесной связанности компонентов: если класс сам создает свои зависимости, его сложно тестировать и модифицировать. DI упрощает замену реализаций (например, можно подменить реальный репозиторий на заглушку для тестов) и улучшает читаемость кода, так как зависимости явно объявлены

2. Каковы основные обязанности каждого из трех слоев приложения (Repository, Service, Controller)? Почему такое разделение считается хорошей практикой? Что произойдет, если объединить логику репозитория и контроллера?

- Repository отвечает за работу с данными: выполнение запросов к БД, маппинг результатов в объекты.
- Service содержит бизнес-логику: валидацию, преобразование данных, вызовы к внешним сервисам.
- Controller обрабатывает HTTP-запросы: парсинг входных данных, маршрутизацию, возврат ответов.

Разделение позволяет соблюсти принцип единственной ответственности. Если объединить репозиторий и контроллер, код станет сложнее тестировать, а бизнес-логика окажется смешана с HTTP-обработкой, что затруднит поддержку.

3. Объясните жизненный цикл зависимости в Litestar. Как именно создается и когда уничтожается экземпляр сессии базы данных при обработке одного HTTP-запроса?

Litestar создает зависимости через провайдеры (например, `provide_db_session`). Для каждого запроса создается новая сессия БД, которая передается в репозиторий и сервис. После обработки запроса сессия автоматически закрывается (благодаря `async with` и `yield` в провайдере). Это гарантирует, что соединения с БД не остаются открытыми.

4. Что такое `async/await` и зачем они используются в данном приложении? Как асинхронность влияет на производительность при работе с базой данных?

Ключевые слова `async/await` позволяют выполнять операции без блокировки потока. Например, пока приложение ждет ответ от БД, оно может обрабатывать другие запросы. В контексте работы с базой данных это повышает производительность, особенно при большом количестве одновременных подключений, так как поток не простаивает в ожидании I/O-операций.

5. Почему в сигнатурах методов `UserRepository` первым аргументом передается `session`? Почему бы не создать его внутри репозитория? Кто и когда должен вызывать `session.commit()` или `session.rollback()`?

Сессия передается как аргумент, чтобы обеспечить атомарность операций в рамках одного запроса. Если создавать сессию внутри репозитория, невозможно будет объединить несколько операций в одну транзакцию. `session.commit()` или `session.rollback()` должен вызываться на уровне, который управляет транзакцией, обычно в сервисе или контроллере, чтобы контролировать границы бизнес-логики.

6. Для чего в методе `get_by_filter` используется пагинация (`count` и `page`)?

Пагинация (параметры `count` и `page`) решает проблему нагрузки на БД и сеть при большом количестве данных. Без пагинации запрос на тысячи записей мог бы замедлить работу приложения или привести к исчерпанию памяти. Ограничение количества результатов делает ответ предсказуемым и эффективным.

7. В текущей реализации `UserService` практически не содержит бизнес-логики и является "прокси" для `UserRepository`. Приведите пример конкретной бизнес-логики (например, проверка уникальности email, хеширование пароля, отправка приветственного письма), которую можно было бы добавить в сервисный слой.

- Проверка уникальности email перед созданием пользователя.
- Хеширование пароля с использованием алгоритма `bcrypt`.
- Валидация сложности пароля (минимум символов, наличие цифр).
- Отправка приветственного email через интеграцию с почтовым сервисом.
- Аудит действий — запись в лог при создании или изменении пользователя.

8. Какие HTTP-статусы должны возвращать каждый из эндпоинтов (`get`, `post`, `put`, `delete`) в различных сценариях (успех, ошибка, не найден)? Обоснуйте свой выбор.

- `GET /users/{id}`: 200 (успех), 404 (пользователь не найден).
- `POST /users`: 201 (успешное создание), 400 (невалидные данные), 409 (конфликт, например, email уже существует).
- `PUT /users/{id}`: 200 (успешное обновление), 404 (пользователь не найден), 400 (ошибка валидации).

- DELETE /users/{id}: 204 (успешное удаление, нет содержимого), 404 (пользователь не найден).

Вывод:

В ходе выполнения лабораторной работы были успешно освоены принципы Dependency Injection и интеграция SQLAlchemy ORM в веб-приложении на базе фреймворка Litestar. Было разработано полнофункциональное CRUD-приложение для управления пользователями с использованием современных подходов к веб-разработке.

Основным достижением стала реализация трехслойной архитектуры, которая позволила четко разделить ответственность между компонентами приложения.

На уровне репозитория организована работа с базой данных через SQLAlchemy, на сервисном слое реализована бизнес-логика, а в контроллерах настроена обработка HTTP-запросов. Особое внимание уделено настройке Dependency Injection, что обеспечило слабую связанность компонентов и упростило тестирование приложения.

Ссылка на репозиторий:

<https://github.com/orfimorf/DevelopmentURFU2025>